

Real-Time Streaming with Kafka & PySpark

Offsets, watermarking, trigger modes and exactly-once Delta writes

An end-to-end guide to Structured Streaming against Kafka: how offsets and consumer groups really behave under Spark, parsing and evolving event schemas, watermarking and late data, stateful aggregations and stream-stream joins, the trigger modes and when each is right, and how checkpoints plus Delta give you exactly-once results on top of at-least-once delivery.

Vivek Pandey — Data Engineer
Azure · Databricks · PySpark · Delta Lake
vp171097.github.io/DevWebsite

Contents

1. The streaming model: micro-batch as an unbounded table
2. Kafka fundamentals for Spark users
3. Reading from Kafka
4. Offsets, checkpoints and delivery semantics
5. Parsing payloads and schema handling
6. Event time, watermarking and late data
7. Windowed aggregations
8. Stateful processing and state stores
9. Stream-stream and stream-static joins
10. Trigger modes
11. Writing to Delta: idempotence and foreachBatch
12. Monitoring and operations

1. The streaming model: micro-batch as an unbounded table

Structured Streaming treats a stream as a table that keeps growing. You write the same DataFrame code you would write for a batch query, and the engine converts it into an incremental plan executed over small batches. That is why a streaming job and its batch equivalent can share transformation code.

Each micro-batch: read the new offset range, compute the incremental result, update state, write the output, then commit progress to the checkpoint. The commit is what makes the run resumable.

Mode	Latency	Notes
Micro-batch (default)	Hundreds of ms to seconds	Full Spark SQL support; the right default
Continuous processing	~1 ms	Experimental, at-least-once only, limited operations

2. Kafka fundamentals for Spark users

Concept	What it is	Why it matters in Spark
Topic	A named stream of records	One source per readStream
Partition	An ordered, append-only log	The unit of parallelism — Spark assigns one task per partition
Offset	A record's position in a partition	Spark tracks these itself, in its checkpoint
Consumer group	A set of cooperating consumers	Spark does NOT use it for offset management
Retention	How long records are kept	If a job is down longer than this, data is lost

Parallelism ceiling A streaming query can use at most one task per Kafka partition. A 6-partition topic will never use more than 6 cores for the read, no matter how large the cluster. If ingestion is the bottleneck, add Kafka partitions — or repartition immediately after reading, which costs a shuffle but unlocks the cluster for the transform.

3. Reading from Kafka

```
raw = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", BROKERS)
    .option("subscribe", "orders,payments")           # or subscribePattern "orders.*"
    .option("startingOffsets", "latest")              # "earliest" on first run to backfill
    .option("maxOffsetsPerTrigger", 500000)           # cap batch size -> stable batch duration
    .option("failOnDataLoss", "true")                 # fail loudly if offsets aged out
    .load())
```

startingOffsets applies only when no checkpoint exists; afterwards the checkpoint always wins.

Kafka column	Type	Use
key	binary	Partitioning key; cast to string
value	binary	The payload — cast and parse
topic	string	Source topic when subscribing to several
partition	int	Diagnosing skew across partitions
offset	long	Position; useful for debugging and dedupe
timestamp	timestamp	Broker ingestion time — not your event time

Broker time is not event time The **timestamp** column is when Kafka received the record. If you window on it you are measuring your pipeline, not the business event. Parse the event's own timestamp out of the payload and watermark on that.

4. Offsets, checkpoints and delivery semantics

Spark does not commit offsets back to Kafka's consumer-group mechanism. It records the offset range of each batch in its own checkpoint directory, which is why two queries with different checkpoints read the same topic independently, and why deleting a checkpoint restarts a stream from scratch.

```
checkpoint/
████ offsets/      # planned offset range per batch, written BEFORE processing
████ commits/      # written AFTER the batch succeeds
████ sources/      # source metadata
████ state/        # keyed state for stateful operators
████ metadata/     # query id – identity of the stream
```

Recovery compares the two directories: an entry in **offsets/** with no matching entry in **commits/** is a batch that was interrupted, and it is replayed. That replay is why the sink must tolerate seeing a batch twice.

Guarantee	How it is achieved
At-most-once	Rarely wanted; requires discarding on failure
At-least-once	Checkpoint replay alone — duplicates are possible
Exactly-once (effectively)	Checkpoint replay + an idempotent sink. Delta provides this via txnAppId/txnVersion or MERGE

The checkpoint is part of your production state One checkpoint per query, never shared. Changing the query in incompatible ways (adding a stateful operator, changing the aggregation keys) invalidates it and requires a new checkpoint plus a reprocessing plan. Back-ups of a checkpoint are not restorable across Spark versions in general.

5. Parsing payloads and schema handling

```

from pyspark.sql import functions as F
from pyspark.sql.types import (StructType, StructField, StringType,
                               TimestampType, DecimalType)

# Declare the schema explicitly. Inference on a stream is unsafe: it samples
# one batch and then fails when batch two looks different.
order_schema = StructType([
    StructField("order_id",      StringType(),      False),
    StructField("customer_id",   StringType(),      True),
    StructField("status",        StringType(),      True),
    StructField("amount",        DecimalType(12, 2), True),
    StructField("event_time",    TimestampType(),   True),
])

parsed = (raw
    .select(F.col("key").cast("string").alias("kafka_key"),
           F.from_json(F.col("value").cast("string"), order_schema).alias("d"),
           F.col("timestamp").alias("kafka_ts"),
           F.col("topic"), F.col("partition"), F.col("offset"))
    .select("kafka_key", "kafka_ts", "topic", "partition", "offset", "d.*"))

```

from_json returns NULL for an unparseable payload rather than failing the batch.

```

# Route unparseable records to a dead-letter table instead of losing them.
tagged = raw.withColumn("parsed",
    F.from_json(F.col("value").cast("string"), order_schema))

good = tagged.filter(F.col("parsed").isNotNull()).select("parsed.*")
bad  = (tagged.filter(F.col("parsed").isNull())
    .select(F.col("value").cast("string").alias("raw_payload"),
           "topic", "partition", "offset",
           F.current_timestamp().alias("_failed_at")))

```

Schema registry With Avro or Protobuf, use **from_avro** / **from_protobuf** with the registry rather than hand-maintained schemas — the registry enforces compatibility at the producer, which is the only place it can actually be enforced.

6. Event time, watermarking and late data

A watermark tells the engine how late an event may be and still be counted. It is a trade: a longer watermark accepts more late data and holds more state; a shorter one is cheaper and silently drops stragglers.

```

# "We will wait up to 10 minutes past the highest event time seen."
windowed = (parsed
    .withWatermark("event_time", "10 minutes")
    .groupBy(F.window("event_time", "5 minutes"), "status")
    .agg(F.count("*").alias("orders"),
        F.sum("amount").alias("amount")))

```

The watermark advances to *max event time seen – delay threshold*. Windows entirely older than the watermark are finalised, emitted, and their state is dropped. Events arriving after that are ignored.

Output mode	Emits	Works with
append	Rows that are final and will not change	Aggregations with a watermark; the usual choice
update	Only rows changed in this batch	Aggregations, with or without a watermark
complete	The entire result table, every batch	Aggregations only; state grows without bound

Unbounded state A streaming aggregation without a watermark keeps every key forever. It runs fine for a week and then fails on state-store size. Any aggregation intended to run indefinitely needs a watermark.

```
# Count what the watermark discarded – otherwise late data vanishes silently.
query.lastProgress["stateOperators"][0]["numRowsDroppedByWatermark"]
```

7. Windowed aggregations

Window	Definition	Example
Tumbling	Fixed, non-overlapping	<code>window(t, '5 minutes')</code>
Sliding	Fixed size, overlapping	<code>window(t, '10 minutes', '5 minutes')</code>
Session	Gap-delimited, variable length	<code>session_window(t, '30 minutes')</code>

```
# Sliding: a 1-hour view refreshed every 10 minutes (each event lands in 6 windows)
sliding = (parsed.withWatermark("event_time", "15 minutes")
    .groupBy(F.window("event_time", "1 hour", "10 minutes"), "status")
    .agg(F.sum("amount").alias("revenue")))

# Session: group activity until a 30-minute gap
sessions = (parsed.withWatermark("event_time", "1 hour")
    .groupBy(F.session_window("event_time", "30 minutes"), "customer_id")
    .agg(F.count("*").alias("events"),
        F.min("event_time").alias("started"),
        F.max("event_time").alias("ended")))

# Flatten the struct for a readable sink
out = sliding.select(F.col("window.start").alias("window_start"),
    F.col("window.end").alias("window_end"),
    "status", "revenue")
```

8. Stateful processing and state stores

Aggregations, deduplication, stream-stream joins and custom stateful operators all keep state between batches. State lives in the state store, is checkpointed, and is the usual cause of a stream that degrades over days rather than failing outright.

```
# Deduplicate with bounded state: the watermark lets old keys expire.
deduped = (parsed
    .withWatermark("event_time", "24 hours")
    .dropDuplicates(["order_id", "event_time"]))

# Without a watermark, dropDuplicates remembers every key forever.
```

dropDuplicatesWithinWatermark (Spark 3.5+) expires keys purely by the watermark, which is usually what you want.

```
# RocksDB keeps state off-heap – the right choice for large keyspaces
spark.conf.set("spark.sql.streaming.stateStore.providerClass",
    "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider")

# Cap how long a single batch may run so a state blow-up surfaces quickly
spark.conf.set("spark.sql.streaming.noDataMicroBatchesEnabled", "true")
```

Watch state size, not just latency In `lastProgress`, track `stateOperators[].numRowsTotal` and `memoryUsedBytes`. Steady growth means a missing or too-generous watermark, and the job will eventually fail during a batch rather than at start-up.

9. Stream-stream and stream-static joins

```
# Stream-static: the static side is re-read each batch. Ideal for dimensions.
customers = spark.read.table("portfolio.ecom_silver.customers") # static
enriched = parsed.join(F.broadcast(customers), "customer_id", "left")
```

```
# Stream-stream: BOTH sides need watermarks AND a time constraint,
# otherwise the engine must buffer both streams indefinitely.
orders_s = order_stream.withWatermark("order_time", "1 hour")
payments_s = payment_stream.withWatermark("payment_time", "2 hours")

joined = orders_s.join(
    payments_s,
    F.expr(
        "order_id = payment_order_id AND "
        "payment_time >= order_time AND "
        "payment_time <= order_time + INTERVAL 1 HOUR"
    ),
    "leftOuter")
```

For an outer join the engine must wait until the watermark proves no match can still arrive — so unmatched rows are emitted late by design.

Join type	Supported	Requirement
inner	Yes	Watermarks recommended to bound state
leftOuter / rightOuter	Yes	Watermark on both sides + time-range condition
fullOuter	Yes (3.1+)	Watermark on both sides + time-range condition
Stream-static left	Yes	Static side on the right
Stream-static right	No	Not supported

10. Trigger modes

Trigger	Behaviour	Use for
processingTime='30 seconds'	Fire on a fixed interval	Steady low-latency pipelines
availableNow=True	Process all available data in batches, then stop	Scheduled incremental jobs
once=True	One batch, then stop (deprecated)	Replaced by availableNow
continuous='1 second'	Long-running, ~1 ms latency	Experimental; very limited operations
(default)	Next batch as soon as the last finishes	Throughput-oriented backfills

availableNow is usually the right answer It gives streaming's incremental bookkeeping — checkpoints, exactly-once, no manual high-water marks — on a schedule, with a cluster that shuts down between runs. Most 'streaming' requirements are actually this.

11. Writing to Delta: idempotence and foreachBatch

```
# Plain append: checkpoint + Delta gives exactly-once for appends.
(deduped.writeStream
  .format("delta")
  .outputMode("append")
  .option("checkpointLocation", f"{CHECKPOINT}/silver_orders")
  .trigger(availableNow=True)
  .toTable("portfolio.ecom_silver.orders"))
```

```
# Upserts require foreachBatch: MERGE is a batch operation.
def upsert(micro_batch_df, batch_id):
    (DeltaTable.forName(spark, "portfolio.ecom_silver.orders")
     .alias("t")
     .merge(micro_batch_df.alias("s"), "t.order_id = s.order_id")
     .whenMatchedUpdateAll(condition="s.event_time > t.event_time")
     .whenNotMatchedInsertAll()
     .execute())

(parsed.writeStream
 .foreachBatch(upsert)
 .option("checkpointLocation", f"{CHECKPOINT}/silver_upsert")
 .trigger(processingTime="1 minute")
 .start())
```

foreachBatch can be called more than once for the same batch_id after a failure — the logic inside must be idempotent. MERGE on a stable key is.

```
# Writing the same micro-batch to two sinks: make each write idempotent
# with the transactional-append options, keyed on batch_id.
def write_both(df, batch_id):
    df.persist()
    (df.write.format("delta").mode("append")
     .option("txnAppId", "orders_pipeline").option("txnVersion", batch_id)
     .saveAsTable("portfolio.ecom_silver.orders"))
    (df.groupBy("status").count().write.format("delta").mode("append")
     .option("txnAppId", "orders_metrics").option("txnVersion", batch_id)
     .saveAsTable("portfolio.ecom_ops.status_counts"))
    df.unpersist()
```

Delta ignores a write whose (txnAppId, txnVersion) it has already committed — that is what makes the replay safe.

12. Monitoring and operations

```
p = query.lastProgress

p["batchId"]                # monotonic batch counter
p["inputRowsPerSecond"]     # ingestion rate
p["processedRowsPerSecond"] # processing rate
p["durationMs"]["triggerExecution"]
p["sources"][0]["endOffset"] # where the stream has reached
p["stateOperators"][0]["numRowsTotal"] # state size – watch this trend
p["stateOperators"][0]["numRowsDroppedByWatermark"]
```

Symptom	Likely cause	Action
Growing Kafka consumer lag	<code>processedRows < inputRows</code>	Raise parallelism or cap <code>maxOffsetsPerTrigger</code>
Batch duration creeping up	State growing unbounded	Add or tighten the watermark; move to RocksDB
Many small files in the sink	Short trigger interval	Lengthen the trigger; enable auto-compaction
Duplicates downstream	Non-idempotent sink logic	MERGE on a key, or <code>txnAppld/txnVersion</code>
Query fails after a restart	Incompatible query change	New checkpoint + a documented reprocess
Data loss on restart	Offsets aged out of retention	Increase Kafka retention; keep <code>failOnDataLoss=true</code>

```
# Alert on lag rather than on failure: a stalled stream rarely throws.
from pyspark.sql.streaming import StreamingQueryListener

class LagListener(StreamingQueryListener):
    def onQueryProgress(self, event):
        p = event.progress
        if p.numInputRows and p.processedRowsPerSecond < p.inputRowsPerSecond * 0.8:
            log.warning("falling behind: batch=%s in=%.0f/s out=%.0f/s",
                        p.batchId, p.inputRowsPerSecond, p.processedRowsPerSecond)

spark.streams.addListener(LagListener())
```

Restart discipline Keep the checkpoint path in configuration, never derived from a timestamp or notebook name. A stream whose checkpoint path changes on redeploy silently reprocesses from the starting offset — which looks like duplicate data to everyone downstream.